

HIGH_LEVEL_ARCHITECTURE

Helix OTA — High-Level Architecture

Overview

This diagram illustrates the top-level system architecture of the Helix OTA platform. The **Helix OTA Server** is the central component that orchestrates update campaigns, serves artifacts, and manages device state. It is supported by three data stores — **PostgreSQL** for persistent relational data, **Redis** for caching and real-time state, and **MinIO** (S3-compatible) for binary artifact storage. Operators interact with the system through the **Dashboard** (web UI), while **Android Devices** communicate with the server over HTTPS to check for, download, and report on updates.

Diagram

```
graph TB
    subgraph Operator["👤 Operator"]
        Dashboard["🖥️ Helix OTA Dashboard<br/>(Next.js / React)"]
    end

    subgraph ServerCluster["☁️ Helix OTA Server Cluster"]
        API["🔧 API Server<br/>(Go / Gin)"]
        Scheduler["🕒 Rollout Scheduler<br/>(Go goroutine)"]
        Notifier["📢 Notification Service<br/>(WebSocket / MQTT)"]
    end

    subgraph DataLayer["💾 Data Layer"]
        PG["🐘 PostgreSQL 16<br/>— Devices, Campaigns,<br/>Rollouts, Artifacts,<br/>Reports, Users"]
        Redis["⚡ Redis 7<br/>— Device heartbeat cache,<br/>Rollout state,<br/>Rate limiting,<br/>Session store"]
        MinIO["📦 MinIO (S3-Compatible)<br/>— OTA update artifacts,<br/>Delta packages,<br/>Full images"]
    end

    subgraph DeviceFleet["📱 Device Fleet (Android)"]
        D1["Device A<br/>Android 14"]
        D2["Device B<br/>Android 14"]
        D3["Device N<br/>Android 13"]
    end
```

end

Dashboard -- "REST / GraphQL
:8080" --> API
API -- "SQL
:5432" --> PG
API -- "GET/SET
:6379" --> Redis
API -- "PUT/GET
:9000" --> MinIO
Scheduler -- "polls & updates
rollout state" --> PG
Scheduler -- "device targeting
cache" --> Redis
Notifier -- "push events" --> Dashboard

D1 -- "HTTPS
:443/api/v1/check" --> API
D2 -- "HTTPS
:443/api/v1/check" --> API
D3 -- "HTTPS
:443/api/v1/check" --> API
D1 -- "HTTPS
:443/api/v1/report" --> API
D2 -- "HTTPS
:443/api/v1/report" --> API
D3 -- "HTTPS
:443/api/v1/report" --> API

API -- "presigned URL
artifact download" --> MinIO
D1 -. "HTTPS GET
(presigned URL)" .-> MinIO
D2 -. "HTTPS GET
(presigned URL)" .-> MinIO
D3 -. "HTTPS GET
(presigned URL)" .-> MinIO

API -- "device online status
heartbeat TTL" --> Redis
API -- "campaign & rollout
CRUD + queries" --> PG
API -- "artifact metadata
& versions" --> PG
API -- "artifact binary
upload / retrieve" --> MinIO

style Dashboard fill:#4FC3F7,stroke:#0277BD,color:#000
style API fill:#66BB6A,stroke:#2E7D32,color:#000
style Scheduler fill:#81C784,stroke:#388E3C,color:#000
style Notifier fill:#81C784,stroke:#388E3C,color:#000
style PG fill:#42A5F5,stroke:#1565C0,color:#fff
style Redis fill:#EF5350,stroke:#C62828,color:#fff
style MinIO fill:#FFA726,stroke:#E65100,color:#000
style D1 fill:#CE93D8,stroke:#6A1B9A,color:#000
style D2 fill:#CE93D8,stroke:#6A1B9A,color:#000
style D3 fill:#CE93D8,stroke:#6A1B9A,color:#000

Component Summary

Component	Technology	Purpose
API Server	Go + Gin	REST API for devices and dashboard; authentication, artifact management, campaign control
Rollout Scheduler	Go (goroutine)	Periodically evaluates active campaigns and selects devices for the next rollout stage
Notification Service	WebSocket / MQTT	

Component	Technology	Purpose
		Pushes real-time rollout progress and device status events to the dashboard
Dashboard	Next.js / React	Web UI for operators to create campaigns, monitor rollouts, and manage devices
PostgreSQL	PostgreSQL 16	Primary data store — all persistent entities (devices, campaigns, artifacts, reports, users, audit log)
Redis	Redis 7	In-memory store — device heartbeat/online cache, rollout state machine, rate limiting, session tokens
MinIO	MinIO (S3-compatible)	Object store — OTA update artifacts (.zip, .img, A/B payload), delta packages, full images
Android Devices	Helix OTA Client (C++)	On-device daemon that checks, downloads, verifies, installs (via <code>update_engine</code>), and reports results

Key Data Flows

1. **Operator creates campaign** → Dashboard → API Server → PostgreSQL (persist) → Scheduler picks up
2. **Device checks for updates** → API Server → Redis (online heartbeat) → PostgreSQL (eligible campaigns) → responds with update info or no-update
3. **Device downloads artifact** → API Server generates presigned MinIO URL → Device downloads directly from MinIO
4. **Device reports result** → API Server → PostgreSQL (report record) → Redis (update device cache) → Dashboard (via notifier)
5. **Scheduler advances rollout** → Scheduler → PostgreSQL (update rollout stage) → Notifier → Dashboard